A PROJECT REPORT ON

TOMATO LEAF DISEASE DETECTION AND PRESCRIPTION SYSTEM

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

RUTVIJ DEO	Exam No: B400050369
HARSHADA JADHAV	Exam No: B400050418
ADITYA LONDHE	Exam No: B400050680
BHUSHAN THOMBARE	Exam No: B400050613

DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025

CERTIFICATE

This is to certify that the project report entitled

TOMATO LEAF DISEASE DETECTION AND PRESCRIPTION SYSTEM

Submitted by

RUTVIJ DEO	Exam No: B400050369
HARSHADA JADHAV	Exam No: B400050418
ADITYA LONDHE	Exam No: B400050680
BHUSHAN THOMBARE	Exam No: B400050313

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. P. S. Joshi** and it is approved for the fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. P. S. Joshi
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date: 12/05/2024

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ***TOMATO LEAF DISEASE DETECTION AND PRESCRIPTION SYSTEM'****.*

We would like to thank our project guide ***Prof. P. S. Joshi*** *for giving us all the help and guidance we needed. We are really grateful to them for their kind support and valuable suggestions.*

We are also grateful to ***Dr G. V. Kale****, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to* ***Dr. A. R. Deshpande****, BE project coordinator, for her timely guidance and support.*

Rutvij Deo
Harshada Jadhav
Aditya Londhe
Bhushan Thombare
(B.E. Computer Engg.)

ABSTRACT

This project, "Tomato Leaf Disease Detection and Prescription," presents a technology-driven solution to this issue. Leveraging the power of image processing and convolutional neural networks (CNNs), the system can accurately identify tomato leaf diseases from images and provide corresponding treatment recommendations. The model, trained on a diverse dataset, achieves an accuracy of approximately 97%, demonstrating both reliability and scalability.

By integrating disease detection with practical prescriptions, the system empowers farmers to take timely action, reducing crop losses and improving overall productivity. This approach underscores the potential of AI-based tools in transforming conventional farming practices and contributing to sustainable agricultural development. *Tomato crops play a crucial role in global agriculture, yet they remain highly susceptible to a range of leaf diseases that can significantly impact both yield and quality. Traditional detection methods are largely manual, often requiring expert intervention, which may not be accessible to every farmer—especially in remote or underserved regions.*

This project, "Tomato Leaf Disease Detection and Prescription," presents a technology-driven solution to this issue. Leveraging the power of image processing and convolutional neural networks (CNNs), the system can accurately identify tomato leaf diseases from images and provide corresponding treatment recommendations. The model, trained on a diverse dataset, achieves an accuracy of approximately 97%, demonstrating both reliability and scalability.

By integrating disease detection with practical prescriptions, the system empowers farmers to take timely action, reducing crop losses and improving overall productivity. This approach underscores the potential of AI-based tools in transforming conventional farming practices and contributing to sustainable agricultural development.

Contents

List of Figures

List of Tables

CHAPTER 1
INTRODUCTION

1.1 Overview

This project centers on developing an intelligent and accessible tool for detecting tomato leaf diseases and providing treatment recommendations. Using a web-based interface, the system allows users—primarily farmers—to upload images of affected tomato leaves. These images are processed through a trained CNN model that identifies the disease class and suggests specific remedies or pesticides accordingly.

Designed with a focus on usability and performance, the system is built using a Python backend with integration into a Streamlit-based application. It supports real-time disease classification and prescription generation, making it a practical resource for users with limited technical background. This solution aims not only to improve early disease detection but also to enhance farm-level decision-making in an efficient, cost-effective manner.

1.2 Motivation

Tomato crops are among the most widely cultivated and consumed vegetables globally. However, they are highly vulnerable to leaf diseases, which can spread rapidly and severely reduce yields. Farmers—especially smallholders—often rely on outdated or observational techniques that are subjective and error-prone, which delays treatment and results in significant economic losses.

This project is motivated by the need to bridge that gap using modern, AI-driven tools. Tomatoes were chosen specifically due to their high commercial value, global significance, and the diverse range of leaf diseases they face. By offering a data-driven disease diagnosis and prescription platform, the system aims to support more sustainable farming practices and improve food security through early intervention and informed treatment.

1.3 Problem Definition and Objectives

Farmers often encounter difficulties in accurately identifying tomato leaf diseases at an early stage. Misidentification or delays in diagnosis can result in improper treatments, further damaging the crop and increasing costs. Moreover, access to agricultural experts is often limited in rural or resource-constrained areas.

This project seeks to address these challenges by developing an automated system capable of detecting diseases through leaf images and suggest-

ing appropriate prescriptions. The goal is to provide a solution that is both accurate and easy to use, requiring minimal technical knowledge from the end user. Ultimately, the system intends to enhance disease management in tomato farming by making timely, informed decisions more accessible.

1.4 Project Scope & Limitations

While the system shows promising results in disease detection and treatment recommendation, there are inherent challenges and limitations that need to be acknowledged. The model's performance can be influenced by the quality of input images, which may vary due to lighting conditions, camera resolution, or background noise. Ensuring robustness across such varied inputs remains a key consideration.

Another limitation is the localization of prescriptions. While the system provides generic treatment recommendations, integrating region-specific agricultural practices or chemical availability would require additional data and customization. Furthermore, in regions with poor internet connectivity or limited access to digital devices, user adoption may be constrained.

Despite these limitations, the project demonstrates the feasibility and value of deploying intelligent disease detection systems in agriculture. Future iterations could focus on expanding the model's capabilities, improving its adaptability, and making it even more accessible to end users.

1.5 Methodology of Problem solving

The development of this tomato leaf disease detection and prescription system followed a structured and iterative methodology. The process began with the collection and curation of a large dataset of tomato leaf images, representing a wide range of common diseases as well as healthy specimens. This dataset formed the foundation for training our machine learning model.

We employed Convolutional Neural Networks (CNNs) due to their strong performance in image classification tasks. The model was trained to detect visual patterns and features associated with specific tomato diseases.

To ensure practical reliability, we conducted tests using real-world images collected from various sources. These real-life testing phases helped us assess the model's robustness under different environmental conditions such as varied lighting and image quality.

In addition to disease detection, the system was designed to provide tailored treatment suggestions using Google's Gemini. These recommendations were based on the classified disease and aligned with commonly accepted agricultural practices.

By adhering to this methodology, the project succeeded in developing a system that is not only accurate but also user-friendly and field-ready for deployment among farming communities.

CHAPTER 2
LITERATURE SURVEY

2.1 Literature survey

The significance of utilizing Convolutional Neural Networks (CNNs) in tomato leaf disease detection lies in their ability to accurately analyze and classify complex patterns within crop images, making them particularly effective for identifying a wide variety of plant diseases. CNNs have been shown to excel in image-based recognition tasks, where they can learn intricate details of leaf textures, colors, and shapes associated with different diseases. By automating the disease detection process, CNNs enable rapid and precise identification, reducing the dependency on manual inspections, which are often slow and unreliable. This approach not only improves detection accuracy but also empowers farmers to make timely decisions based on data-driven insights, ultimately aiding in better crop management and reducing losses due to disease.

Wang, X., Liu, J. An efficient deep learning model for tomato disease detection. Plant Methods 20, 61 (2024)[8].This study introduces a novel method, TomatoDet, designed to improve the accuracy of detecting tomato diseases in complex agricultural settings. The method targets four common tomato diseases—late blight, gray leaf spot, brown rot, and leaf mold—while also distinguishing healthy tomatoes. To enhance detection precision, TomatoDet employs a feature extraction module that incorporates Swin-DDETR's self-attention mechanism, which strengthens the model's ability to identify small disease targets amidst complex backgrounds. Additionally, the Meta-ACON activation function is integrated to amplify disease feature representation, and a modified bidirectional feature pyramid network (IBiFPN) is used for effective multi-scale feature merging, reducing false positives and negatives from overlapping and occluded disease areas.The method achieves a high mean Average Precision (mAP) of 92.3% on a specialized dataset, showing an 8.7% improvement over baseline models, with a detection speed of 46.6 frames per second (FPS). These results underscore TomatoDet's potential to meet the real-time demands of agricultural applications, providing accurate and efficient disease identification in complex environments.

Liu, J., Wang, X. Early recognition of tomato gray leaf spot dis- ease based on MobileNetv2-YOLOv3 model. Plant Methods 16, 83 (2020)[9] This study proposes an early recognition method of tomato leaf spot based on MobileNetv2-YOLOv3 model to achieve a good balance between the accuracy and real-time detection of tomato gray leaf spot. This method improves the accuracy of the regression box of tomato gray leaf spot recognition by

introducing the GIoU bounding box regression loss function. A MobileNetv2-YOLOv3 lightweight network model, which uses MobileNetv2 as the backbone network of the model, is proposed to facilitate the migration to the mobile terminal. The pre-training method combining mixup training and transfer learning is used to improve the generalisation ability of the model. The images captured under four different conditions are statistically analysed. The recognition effect of the models is evaluated by the F1 score and the AP value, and the experiment is compared with Faster-RCNN and SSD models. Experimental results show that the recognition effect of the proposed model is significantly improved. In the test dataset of images captured under the background of sufficient light without leaf shelter, the F1 score and AP value are 94.13% and 92.53%, and the average IOU value is 89.92%. In all the test sets, the F1 score and AP value are 93.24% and 91.32%, and the average IOU value is 86.98%. The object detection speed can reach 246 frames/s on GPU, the extrapolation speed for a single 416×416 picture is 16.9 ms, the detection speed on CPU can reach 22 frames/s, the extrapolation speed is 80.9 ms and the memory occupied by the model is 28 MB.

Zhang, J., Qi, C., Mecha, P. et al. Pseudo high-frequency boosts the generalization of a convolutional neural network for cassava disease detection. Plant Methods 18, 136 (2022)[10]. The paper proposes the ArsenicNetPlus neural network to enhance signal transmission and accuracy in cassava disease detection by addressing signal frequency degradation in convolutional neural networks (CNNs). Frequency downconversion in CNN channels often leads to spatial information loss, and ArsenicNetPlus mitigates this by replenishing Fourier series coefficients to reduce information transmission loss. The model incorporates a multiattention mechanism to maintain long-term dependencies in cassava disease features, employs depthwise convolution to eliminate aliasing signals and manage downconversion before sampling, and utilizes an instance batch normalization algorithm to keep features well-structured across channels. Additionally, the ArsenicPlus block generates pseudo high-frequency signals within the residual structure to enhance information retention. When tested on Cassava Datasets, ArsenicNetPlus outperformed comparative models—V2-ResNet-101, EfficientNet-B5, RepVGG-B3g4, and AlexNet—demonstrating superior accuracy, a loss of 1.2440, and an improved F1-score.

Agarwal, A., Sarkar, A., Dubey, A.K. (2019)[1]. Computer Vision-Based Fruit Disease Detection and Classification. In: Tiwari, S., Trivedi, M., Mishra, K., Misra, A., Kumar, K. (eds) Smart Innovations in Communication and Computational Sciences. Advances in Intelligent Systems and Computing, vol 851. Springer, Singapore . This research paper presents an

automatic method for detecting and classifying apple diseases—apple scab, black rot canker, and core rot—using image processing techniques. The system uses K-means clustering for segmentation, GLCM for feature extraction, and an SVM classifier, achieving 98.387% accuracy. The method is user-friendly and efficient, with potential future applications for other fruits.

Jafar A, Bibi N, Naqvi RA, Sadeghi-Niaraki A and Jeong D (2024)[2] Revolutionizing agriculture with artificial intelligence: plant disease detection methods, applications, and their limitations. Front. Plant Sci. 15:1356260. doi: 10.3389/fpls.2024.1356260 .This study highlights the importance of rapid and accurate plant disease detection for improving crop yields and minimizing economic losses caused by infections. Traditional methods are slow and resource-intensive, leading to the adoption of AI and IoT-based automated detection. Focusing on tomato, chilli, potato, and cucumber, it outlines common diseases, symptoms, and AI techniques like machine learning (ML) and deep learning (DL) for disease prediction. Key steps include image acquisition, preprocessing, segmentation, and classification. The research also reviews existing datasets and models, discusses challenges, and explores future prospects with AI and IoT for efficient field-based monitoring.

Ding, J., Qiao, Y. Zhang, L[3]. Plant disease prescription recommendation based on electronic medical records and sentence embedding retrieval. Plant Methods 19, 91 (2023) [4] The paper proposes a plant disease prescription recommendation method called PRSER based on sentence embedding retrieval. PRSER uses a pre-trained language model and a sentence embedding method with contrast learning ideas to create a semantic matching model and retrieves optimal prescriptions from a constructed prescription reference database. The model achieves high performance in both closed-set and open-set testing, demonstrating its effectiveness and generalization ability for recommending plant disease prescriptions.

Li L, Zhang S, Wang B. “Plant disease detection and classification by deep learning—a review. IEEE Access. 2021;9:56683–98. [4]. This paper reviews recent advancements in plant disease recognition, emphasizing image processing, deep learning (DL), and hyperspectral imaging techniques. Traditional methods of disease detection, relying on expert analysis, are labor-intensive and subjective. Modern approaches using image recognition and DL, particularly convolutional neural networks (CNNs), offer higher accuracy and efficiency, overcoming the limitations of manual feature extraction. However, challenges remain, such as the need for large, diverse datasets and improvements in early detection. The paper also highlights transfer learning

as a solution for small datasets and discusses recent DL models and imaging techniques for identifying plant diseases. The review aims to provide insights into the latest research, methods, and gaps in the field.

Alzubaidi, L., Zhang, J., Humaidi, A.J. et al. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. J Big Data 8, 53 (2021) [5] Deep learning (DL) has become the gold standard in machine learning (ML), outperforming traditional techniques across various domains, including cybersecurity, natural language processing, and medical information processing. This review aims to provide a comprehensive understanding of DL by exploring its concepts, challenges, applications, and computational tools, with a focus on convolutional neural networks (CNNs). By synthesizing over 300 recent research papers, this work addresses existing gaps in the literature and highlights the significance of DL in advancing multiple fields.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

In the development of the Tomato Leaf Disease Detection and Prescription system, several key assumptions and dependencies are essential to the Software Requirements Specification (SRS). It is assumed that the system will rely on machine learning techniques, specifically Convolutional Neural Networks (CNNs), for accurate and efficient image-based crop disease detection. This dependence on CNNs necessitates a high-quality dataset of crop images for training and validation, ensuring the model's ability to recognize a diverse set of crop diseases with accuracy.

Additionally, it is assumed that the target users, primarily farmers, may have limited technical skills. This assumption guides the design towards a user-friendly interface with straightforward navigation, making it accessible to users of varying technical backgrounds. Reliable image capture devices, such as mobile phones or tablets, are also assumed to be accessible to the users, as high-quality images are crucial for accurate disease detection.

The project will depend on external libraries and frameworks for machine learning and image processing, such as TensorFlow or PyTorch, to facilitate CNN model development. Internet connectivity is another key dependency, particularly if the system includes cloud-based components for processing or updating disease recognition models, though an offline mode may be considered. Lastly, access to agricultural databases for disease information and treatment recommendations is assumed, as these will provide users with accurate, localized solutions based on detected diseases.

3.2 Functional Requirements

3.2.1 System Requirements

The system will utilize a Convolutional Neural Network (CNN) to detect crop diseases from uploaded images accurately. Users will be able to upload images of crops through a web interface, where the CNN model, developed and trained in Google Colab, will analyze and classify the image, providing the user with a disease diagnosis. The model will rely on libraries such as TensorFlow and NumPy to support machine learning operations, while Python's Pillow (PIL) library will preprocess images to ensure they are suitable for the CNN model.

To maintain model accuracy, the system is using datasets from Kaggle for training and validation, ensuring the CNN is capable of recognizing various crop diseases under different conditions. The entire diagnostic process will be streamlined to provide users with results in real-time, empowering them to make quick decisions for disease management.

3.2.2 User Interface and Experience Design Requirements

The system's user interface (UI) will be developed using Streamlit, enabling an interactive and accessible web application that allows users to upload crop images for analysis and receive disease detection results and treatment recommendations. The interface will be designed to be intuitive and easy to navigate, especially for users with limited technical experience, such as farmers.

The main page will include options to upload images, view analysis results, and access treatment recommendations. After processing, the system will present the results in a straightforward format, with disease identification, potential causes, and suggested treatments displayed clearly. Visual cues, such as color coding for disease severity, will guide users to understand the information at a glance.

3.3 External Interface Requirements

3.3.1 Hardware Interfaces

The system requires the following hardware for effective development and deployment:

- **Development:**
  - Computer with sufficient processing power and RAM for running Python, TensorFlow, and image processing tasks.
  - GPU for accelerated processing of machine learning/CNN models.
  - Sufficient storage (HDD/SSD) for dataset, model files, and backups.
- **User Side**
  - Mobile devices (Android, iOS) for image upload and interaction with the system.
  - **PC Specifications:**
    - PC with at least 4 GB RAM and a dual-core CPU for smooth performance of the web-based application.

3.3.2 Software Interfaces

The software iterfaces for both development and deployment are as follows:

1. **Development:**
   - Operating System: Windows 10/11, macOS Monterey, or Linux.
   - Google Colab for ease of coding and training models in the cloud.
   - Version Control System: Git for source code management and collaboration.
   - Programming Language: Python.
   - Google Gemini for Prescription.
   - Libraries:
     - TensorFlow, Keras for building and training the CNN models.
     - Pillow for image preprocessing tasks.
     - NumPy for data manipulation and numerical operations.

  - Random for data augmentation during training.
- Cloud Storage: Kaggle for dataset hosting and access.
- Web Deployment: Streamlit for creating an interactive web application for AI-ML deployment.

2. **Deployment:**

   - User Interaction: Web-based application via Streamlit accessible through browsers (Chrome, Firefox, Safari).
   - Internet Requirements: Stable internet connection to upload images and receive feedback.

3.3.3 Other Requirements

- **Testing Tools:** Kaggle and custom scripts for dataset evaluation and model validation. Python unit testing for code quality assurance.
- **Remote Collaboration Tools:** Google Drive for storing datasets and backups. Zoom or Slack for team meetings and collaboration.
- **Backup Solutions:** Cloud storage (e.g., Google Drive, Dropbox) for project backups.

CHAPTER 4
SYSTEM DESIGN

4.1 Dataset Structure

Dataset (New Plant Disease Dataset) consists of about 87K rgb images of healthy and diseased crop leaves which is categorized into 38 different classes in train and valid directories. Among which there are 10 Tomato leaf classes, 9 of which are Infected classes and one Healthy class.

Class Name	**Images**	**Class Name**	**Images**
Apple_Apple_scab	2016	Apple_Black_rot	1987
Apple_Cedar_apple_rust	1760	Apple_healthy	2008
Blueberry_healthy	1816	Cherry_Powdery_mildew	1683
Cherry_healthy	1826	Corn_Cercospora_leaf_spot	1642
Corn_Common_rust	1907	Corn_Northern_Leaf_Blight	1908
Corn_healthy	1859	Grape_Black_rot	1888
Grape_Esca	1920	Grape_Leaf_blight	1722
Grape_healthy	1692	Orange_Haunglongbing	2010
Peach_Bacterial_spot	1838	Peach_healthy	1728
Pepper_Bacterial_spot	1913	Pepper_healthy	1988
Potato_Early_blight	1939	Potato_Late_blight	1939
Potato_healthy	1824	Raspberry_healthy	1781
Soybean_healthy	2022	Squash_Powdery_mildew	1736
Strawberry_Leaf_scorch	1774	Strawberry_healthy	1824
Tomato_Bacterial_spot	1702	Tomato_Early_blight	1920
Tomato_Late_blight	1851	Tomato_Leaf_Mold	1882
Tomato_Septoria_leaf_spot	1745	Tomato_Spider_mites	1741
Tomato_Target_Spot	1827	Tomato_Yellow_Leaf_Curl_Virus	1961
Tomato_mosaic_virus	1790	Tomato_healthy	1926
		Total	**69,377**

Table 4.1: Training dataset

Class Name	**Images**	**Class Name**	**Images**
Apple_Apple_scab	504	Apple_Black_rot	497
Apple_Cedar_apple_rust	440	Apple_healthy	502
Blueberry_healthy	454	Cherry_Powdery_mildew	421
Cherry_healthy	456	Corn_Cercospora_leaf_spot	410
Corn_Common_rust	477	Corn_Northern_Leaf_Blight	477
Corn_healthy	465	Grape_Black_rot	472
Grape_Esca	480	Grape_Leaf_blight	430
Grape_healthy	423	Orange_Haunglongbing	503
Peach_Bacterial_spot	459	Peach_healthy	432
Pepper_Bacterial_spot	478	Pepper_healthy	497
Potato_Early_blight	485	Potato_Late_blight	485
Potato_healthy	456	Raspberry_healthy	445
Soybean_healthy	505	Squash_Powdery_mildew	434
Strawberry_Leaf_scorch	444	Strawberry_healthy	456
Tomato_Bacterial_spot	425	Tomato_Early_blight	480
Tomato_Late_blight	463	Tomato_Leaf_Mold	470
Tomato_Septoria_leaf_spot	436	Tomato_Spider_mites	435
Tomato_Target_Spot	457	Tomato_Yellow_Leaf_Curl_Virus	490
Tomato_mosaic_virus	448	Tomato_healthy	481
		Total	**17,772**

Table 4.2: Validation dataset

4.2 Project Architecture

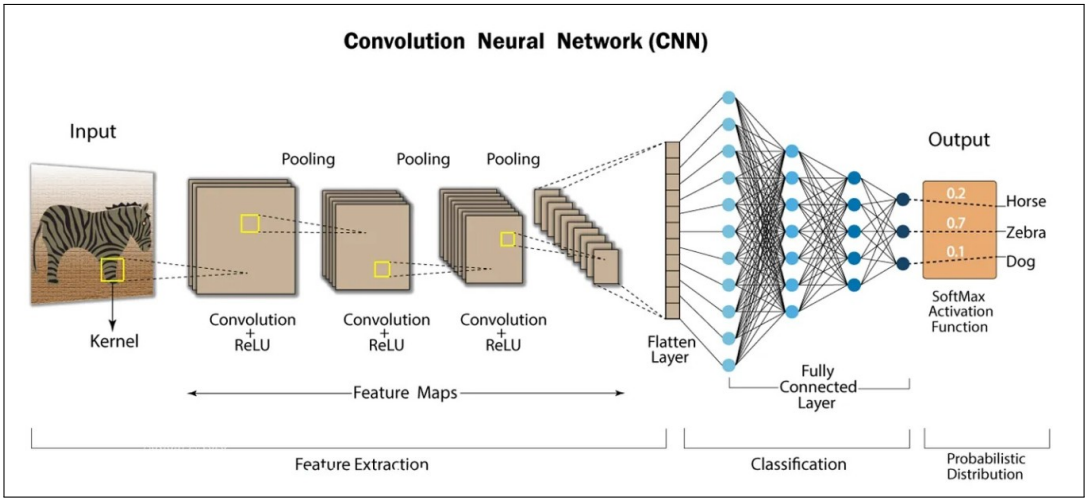


Figure 4.1: CNN Architecture[7]

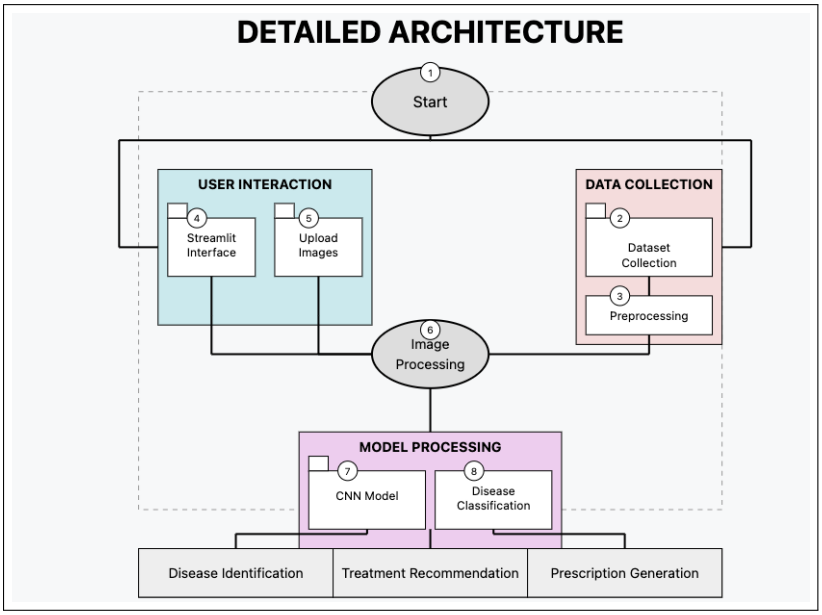


Figure 4.2: System Architecture

4.3 Flow Diagram

The following diagram illustrates the working of the final result.

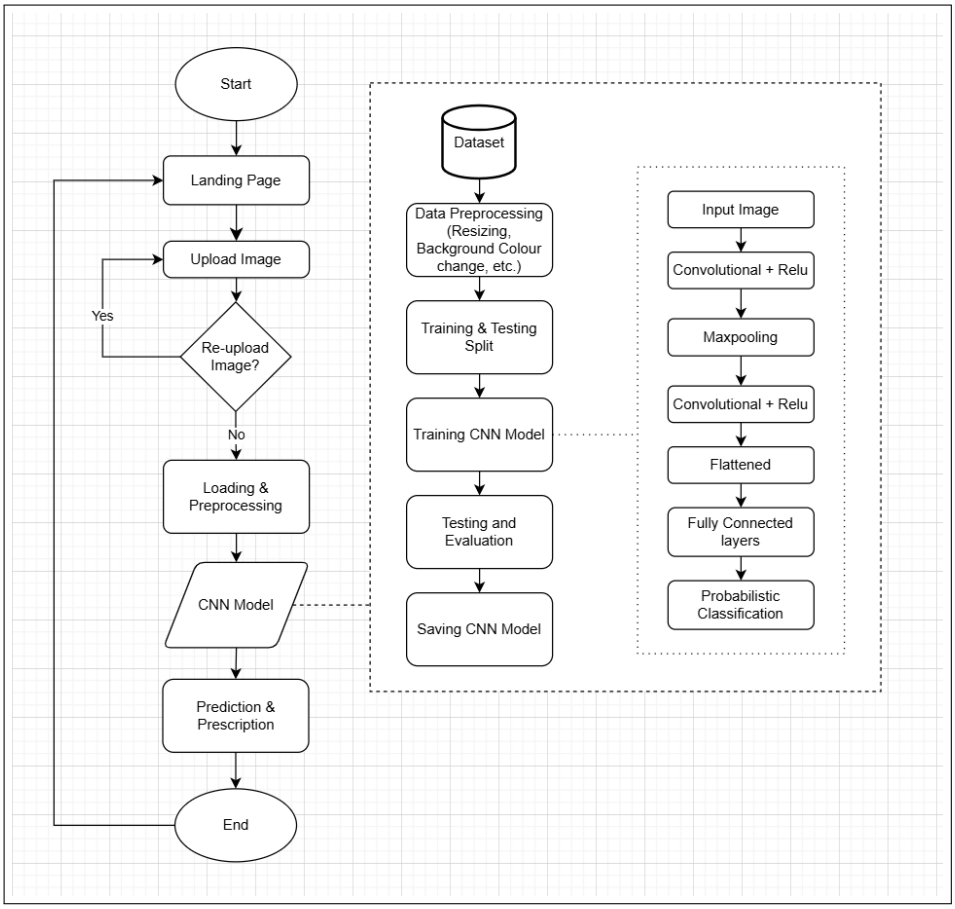


Figure 4.3: Flow Diagram

4.4 Usecase Diagram

The following diagram shows the usecase of the final model.

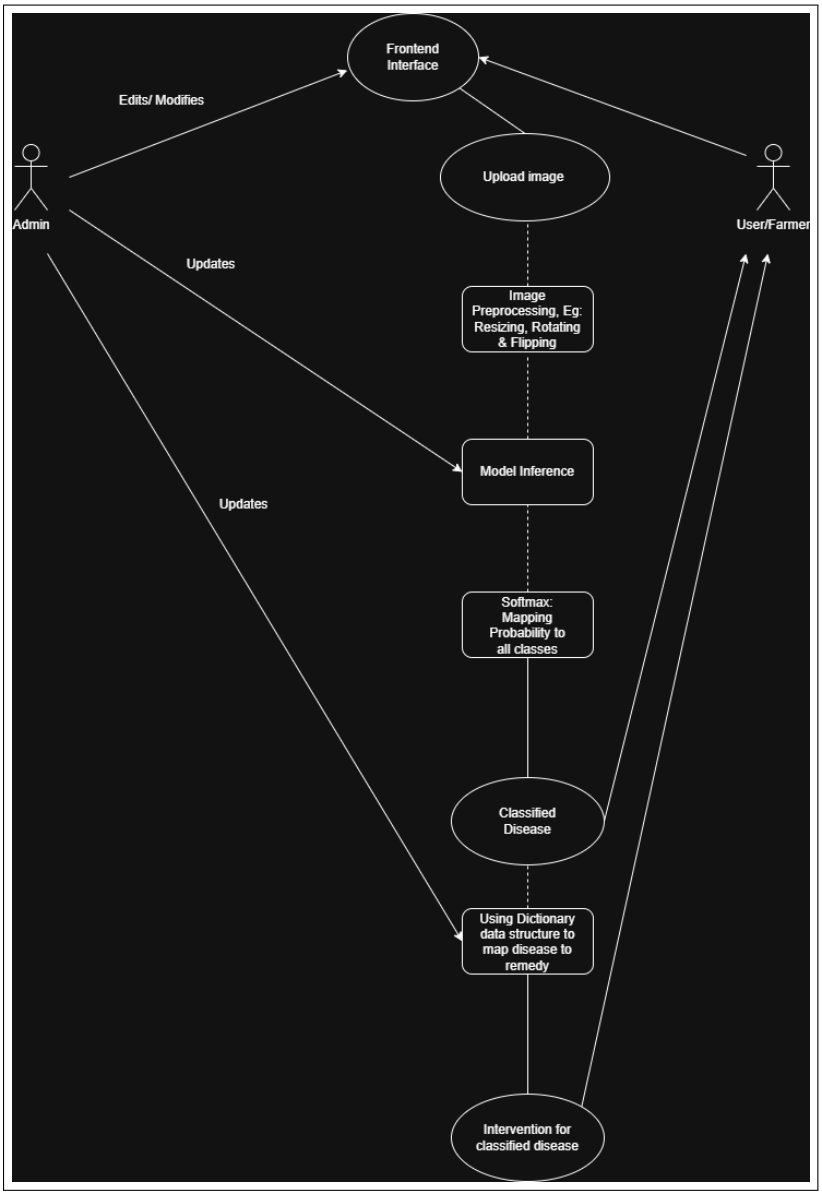


Figure 4.4: Usecase Diagram

CHAPTER 5
PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Dataset Preparation and Preprocessing:** Estimated 2-3 weeks to collect and preprocess a comprehensive dataset of crop images, including both diseased and healthy samples. This phase involves data gathering, applying preprocessing techniques (e.g., normalization, augmentation), and ensuring dataset consistency for quality training and testing. The dataset will cover various crops and disease types.

- **Model Development and Evaluation:** Estimated 3-4 months for the development and evaluation of a machine learning model, focusing on CNNs (convolutional neural networks) for image-based disease detection. This phase includes setting up the coding environment (e.g., Python, TensorFlow), training multiple model prototypes, and validating their performance across subsets of data to ensure accurate and reliable disease detection.

- **Application Integration and Testing:** Estimated 2 months for integrating the trained model into a user-friendly application. This includes designing an intuitive interface, allowing farmers to upload images and receive treatment recommendations. The phase also includes feedback loops to refine detection accuracy and improve user experience.

- **Testing, Validation, and Further Optimization:**Estimated 2 months for comprehensive testing, validation, and further optimization of the application and model. This includes verifying the system's accuracy, ensuring high-quality disease treatment recommendations, and optimizing the user experience under varying conditions, such as different image qualities and lighting.

- **Documentation and Review:** Estimated 2-3 weeks for detailed project documentation, capturing methodologies, challenges, and solutions encountered throughout the project. The final report will summarize all phases, and the findings may be submitted to agricultural technology conferences or explored for potential partnerships to expand the project's reach.

Approximately 8 months, though the actual timeline may vary based on specific project constraints, resource availability, and emerging needs during development.

5.1.2 Project Resources

1. **Machine Learning Engineers (3):** Responsible for developing and optimizing the crop disease detection algorithms. Their main tasks include designing, training, and fine-tuning machine learning models, particularly convolutional neural networks (CNNs), to detect crop diseases accurately. They collaborate with the UI/UX designer to integrate model outputs into a user-friendly interface and adjust the system based on testing feedback.

2. **UI/UX Designers (2):** Responsible for designing an intuitive and accessible user interface for the system, focusing on key user screens such as image upload, disease detection results, and treatment recommendation displays. The UI/UX designer will conduct usability testing, refining the interface to meet the needs of farmers and other end users.

3. **Project Management Tool:** We used asana to organize tasks, set deadlines, and manage project milestones. Asana provides a visual interface for tracking progress, assigning responsibilities, and ensuring each phase of development stays on schedule.

4. **Collaboration Tools:** GitHub for version control and collaborative coding, Google Colab as a primary environment for coding, model training, testing, and sharing development progress and code snippets., Google Meet for team meetings and discussions, and Discord for real-time communication and collaboration among team members.

5.2 Risk Management

5.2.1 Risk Identification

- **Technical Risk:**
    1. Potential challenges in training machine learning models with high accuracy due to limited or diverse image quality in the dataset.
    2. Risks associated with processing large image datasets, which may require high computational resources.
    3. Integration issues between the ML model and the user interface in Streamlit.
    4. Difficulties in ensuring real-time disease detection and prescription accuracy with diverse crop images.
- **Data Risk:** Data acquisition may be limited or inconsistent due to varying crop disease images and conditions, affecting the accuracy of the model.
- **Operational Risk:** Unforeseen changes in project requirements or team availability could impact project timeline and quality.

5.2.2 Risk Analysis

- **Technical Risk:** Assessed as high impact, given the critical role of model accuracy and processing speed. This risk has a medium likelihood. To mitigate, iterative model evaluation and optimization processes are planned.
- **Data Risk:** Evaluated as medium impact and medium likelihood due to potential challenges in accessing diverse, high-quality crop disease images. To mitigate, additional sources for data and preprocessing techniques are considered.
- **Operational Risk:** Considered medium impact with medium likelihood. Effective task scheduling and scope management will address possible disruptions due to scope or personnel changes.

5.2.3 Overview of Risk Mitigation, Monitoring, and Management

- **Technical Risk Mitigation:** Continuous model training, testing, and regular validation using real-world images will be conducted to maintain model accuracy. Streamlined integration between Google Colab and Streamlit will be ensured to avoid compatibility issues.
- **Data Risk Mitigation:** Diverse datasets will be sourced and additional preprocessing techniques (e.g., data augmentation) applied to address data limitations and improve model reliability.
- **Operational Risk Mitigation:** A well-defined project scope and flexible scheduling will help manage any changes in requirements or team dynamics.
- **Risk Monitoring:** Weekly project status meetings will assess identified risks and monitor for new potential risks, enabling timely responses.

5.3 Project Schedule

5.3.1 Project Task Set

- Define the problem statement, clarifying the need for a system to detect crop diseases and provide treatment recommendations.
- Conduct research to outline the scope of the project, including target crops, disease types, and potential impact on farming practices.
- Establish workflows for disease detection and treatment recommendations, identifying potential challenges and devising solutions.
- Set up necessary environments and tools, such as Python, Jupyter Notebook, and deep learning libraries like TensorFlow or PyTorch.
- Configure a version control system for collaborative development and version tracking.
- Review existing image-based disease detection systems to understand methodologies and identify challenges.
- Gather a diverse dataset of crop images, covering various diseases and healthy samples for training and testing purposes.
- Research and apply image preprocessing techniques (e.g., normalization, augmentation) to ensure consistent dataset quality.
- Evaluate machine learning algorithms, such as convolutional neural networks (CNNs), for image-based disease detection. Build initial model prototypes and test them on a subset of data to assess feasibility and initial accuracy.
- Testing and Validation.Integrate the trained model and treatment database into a user-friendly application, enabling farmers to upload images and receive guidance.
- Use feedback to refine detection accuracy, recommendation quality, and user experience, iterating on the design as needed.
- Document the project's methodologies, challenges, solutions, and outcomes in a final report.
- Submit the project to agricultural technology conferences and explore potential partnerships to increase its reach.

5.3.2 Timeline Chart

Figure 5.1 presents an overview of the project timeline, summarizing the schedule from project inception through to the expected completion date.

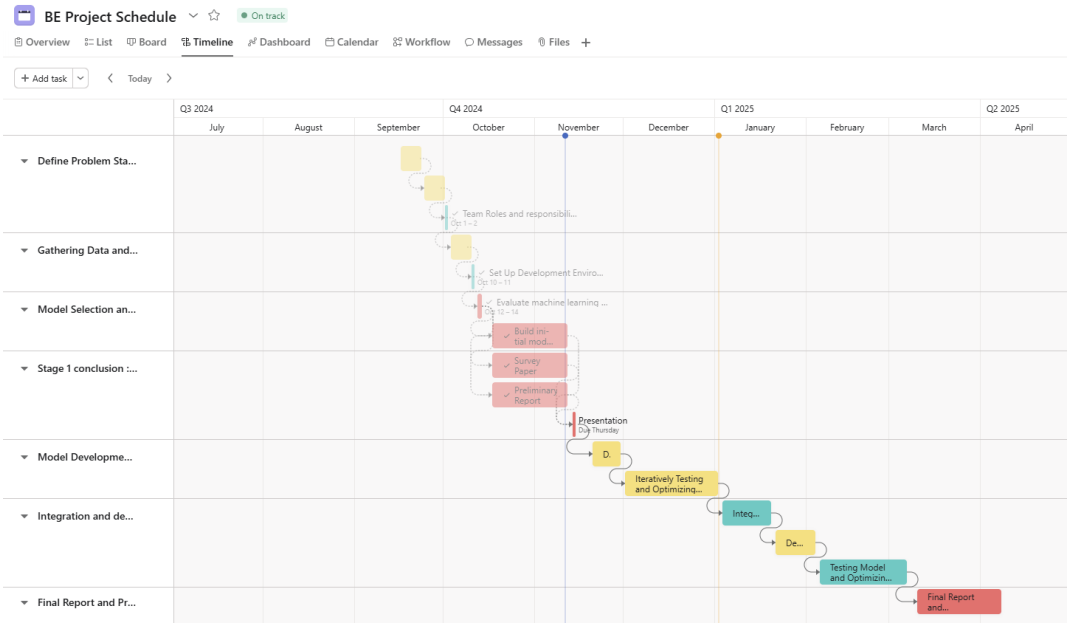


Figure 5.1: Timeline Chart

5.4 Team Organization

5.4.1 Team structure

- **Rutvij Deo:** M.L., Development, Data collection.
- **Harshada Jadhav:** Data collection, documentation.
- **Aditya Londhe:** M.L., Development, Documentation, Data gathering.
- **Bhushan Thombare:** Development, Data collection, documentation.
- **Prof. Pallavi Joshi:** Internal mentor and guide.

CHAPTER 6
PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

- **Model Building and Training:** A Convolutional Neural Network (CNN) was constructed on Kaggle New Plant Disease Dataset using Keras with TensorFlow backend. The model consists of multiple convolutional blocks, followed by dense layers. It was trained for 30 epochs with early stopping and learning rate reduction mechanisms.
- **Model Evaluation and Visualization:** The model was evaluated using accuracy, precision, recall, and loss metrics. Training and validation performance was plotted to visualize the learning trends.
- **Prediction and Visualization Module:** A separate module was created to load a saved model, preprocess user-uploaded images, make predictions, and visually display results along with prediction.
- **User Interface and Prescription with Streamlit and Google Gemini:** A web interface was developed using Streamlit to enable users to interact with the system, upload images, and view disease predictions along with Prescription generated by Gemini.

6.2 Implementation of Model

The following diagrams shows the implementation of the model.

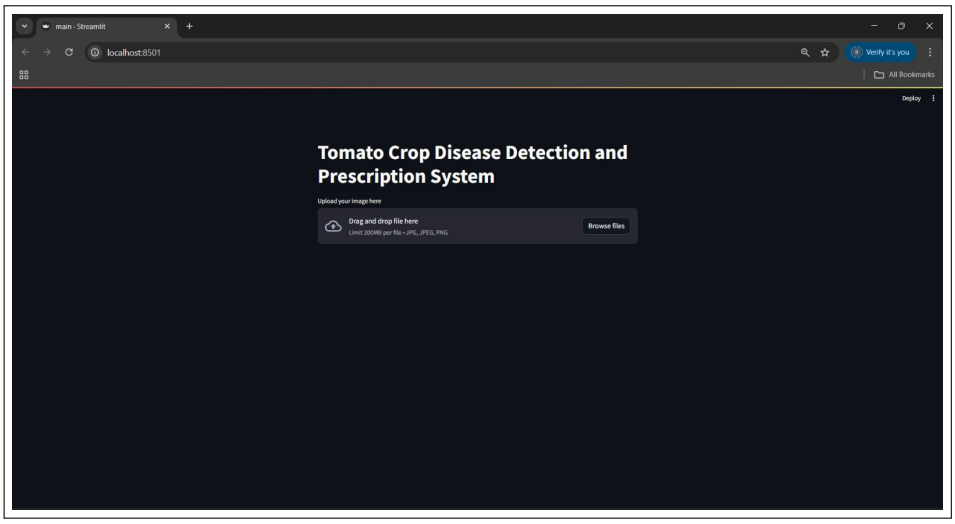


Figure 6.1: Home Page

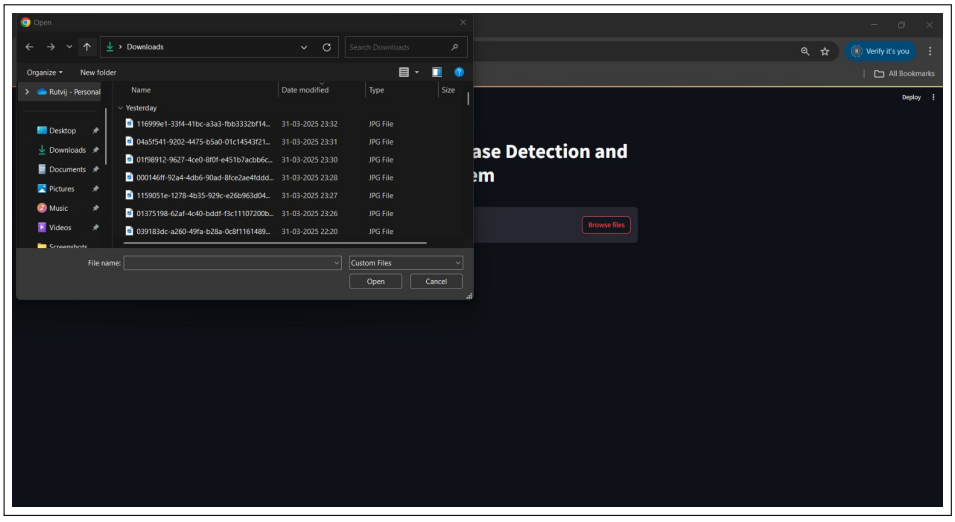


Figure 6.2: Upload Tomato Leaf Image

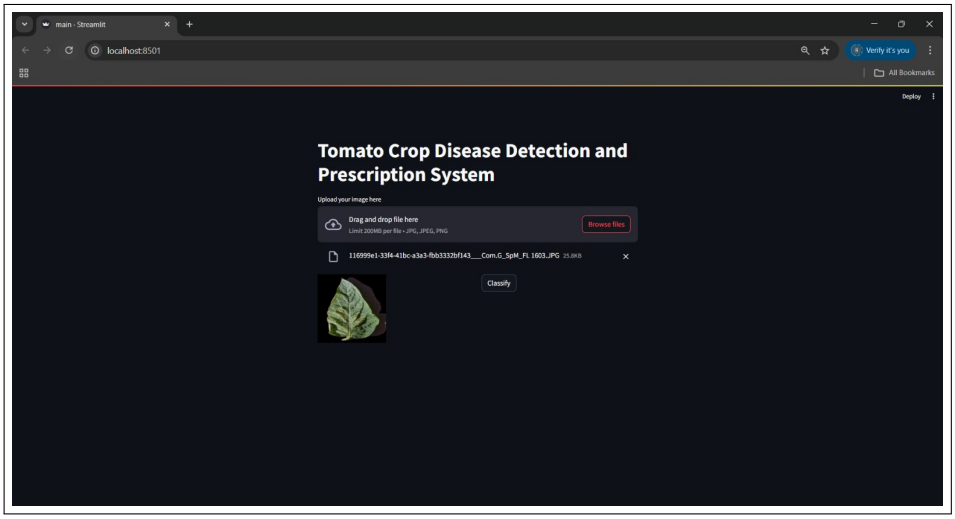


Figure 6.3: Re-upload or proceed to classify the disease

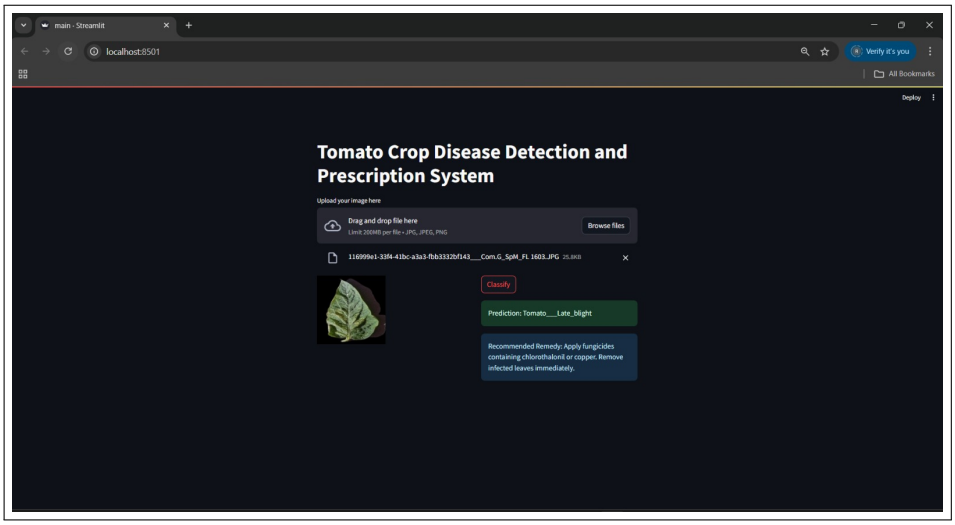


Figure 6.4: Get the info and prescription of the disease

6.3 Tools and Technologies Used

- **Programming Language:** Python 3.10
- **Libraries and Frameworks:** TensorFlow, Keras, NumPy, Matplotlib, Pillow, Streamlit, ImageDataGenerator
- **Development Environment:** Google Colab for model training; Streamlit for deployment
- **Dataset Source:** Kaggle – New Plant Diseases Dataset (Augmented)
- **Model Saving and Loading:** HDF5 format (.h5) for saving and reusing trained models
- **Hardware Used:** Google Colab GPU runtime (Tesla T4), local system with 8 GB RAM (for testing)
- **AI Tools:** Google Gemini To get Prescriptions based on the plant disease.

6.4 Algorithm Details

6.4.1 Algorithm 1: Image Preprocessing Algorithm

This algorithm ensures all input images are standardized before being fed into the neural network. It includes resizing images to a uniform shape (256x256 pixels), converting them to RGB format, and normalizing pixel values to a range between 0 and 1. The ImageDataGenerator class in Keras is used for scaling,rotating, vertical and horizontal flipping images and generating batches of tensor image data with real-time data augmentation during training and validation.

```
# Image Parameters
img_size = 256
# img_size = 256
batch_size = 32


# Image Data Generators
data_gen = ImageDataGenerator(
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True,
    vertical_flip=True,
    shear_range=0.1,
    fill_mode='nearest',
    rescale=1./255
  )
# Train Generator
train_generator = data_gen.flow_from_directory(
    train_dir,  # Use your training directory
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='categorical'
)
val_data_gen = ImageDataGenerator(
    rescale=1./255
  )
# Validation Generator
validation_generator = val_data_gen.flow_from_directory(
    validation_dir,  # Use your validation directory
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='categorical'
)
```

Figure 6.5: Image Preprocessing

6.4.2 Algorithm 2: Convolutional Neural Network (CNN) Model

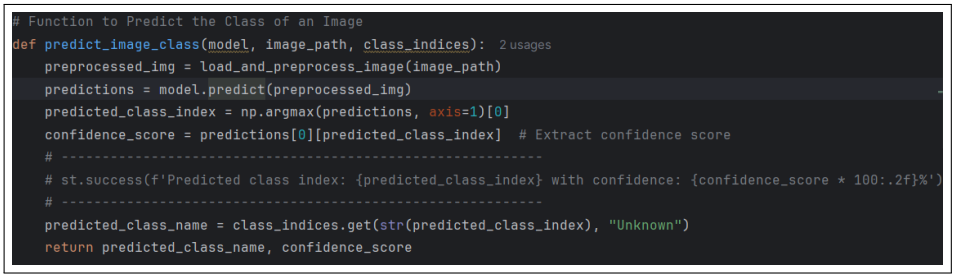
The CNN architecture is designed for automatic feature extraction from input images. It consists of three main convolutional blocks (3 Blocks each with 2 Conv2D, 1 MaxPooling and Dropout=0.25), each including convolution layers with ReLU activation, max pooling layers, and dropout for regularization. These are followed by a flattening layer and two dense (fully connected) layers, with the final layer using a softmax activation function to classify the image into one of the plant disease categories.

```
from tensorflow.keras import regularizers

model = models.Sequential()

# Convolution Block 1
model.add(layers.Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(img_size, img_size, 3)))
model.add(layers.Conv2D(32, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D(pool_size=(2, 2)))
model.add(layers.Dropout(0.25))

# Convolution Block 2
model.add(layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D(pool_size=(2, 2)))
model.add(layers.Dropout(0.25))

# Convolution Block 3
model.add(layers.Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(layers.Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D(pool_size=(2, 2)))
model.add(layers.Dropout(0.25))

# Fully Connected Layers
model.add(layers.Flatten())
model.add(layers.Dense(512, activation='relu', kernel_regularizer=regularizers.l2(0.001)))
model.add(layers.Dropout(0.5))
model.add(layers.Dense(train_generator.num_classes, activation='softmax'))
```

Figure 6.6: Model Code

6.4.3 Algorithm 3: Prediction and Class Mapping

This algorithm is used post-training to classify new images. It loads the trained model and class index mappings from JSON. The uploaded image is preprocessed using the same pipeline as the training images, and then passed through the model to obtain prediction probabilities. The class with the highest probability is selected, and the corresponding class name is displayed along with Prescription generated by Gemini AI.

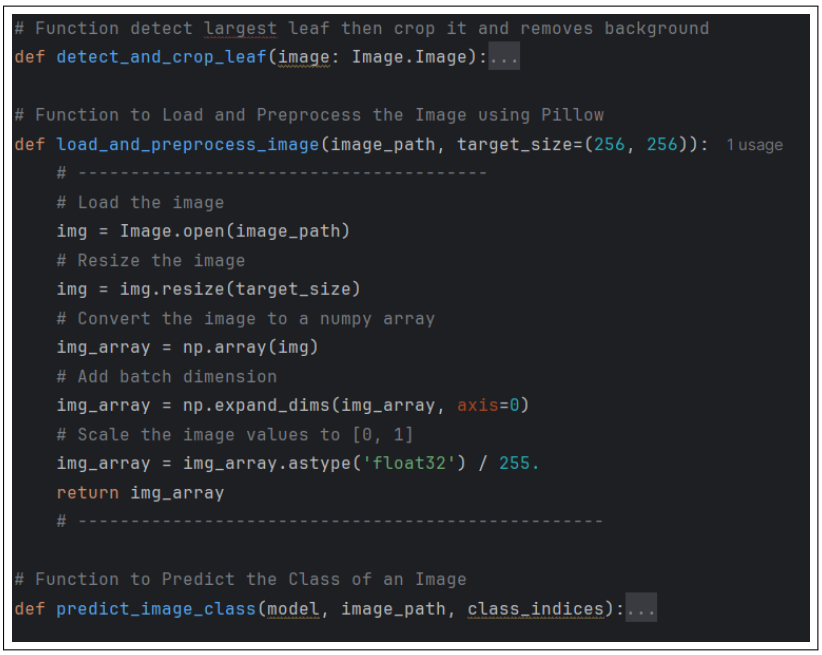


```python
# Function to Predict the Class of an Image
def predict_image_class(model, image_path, class_indices):  2 usages
    preprocessed_img = load_and_preprocess_image(image_path)
    predictions = model.predict(preprocessed_img)
    predicted_class_index = np.argmax(predictions, axis=1)[0]
    confidence_score = predictions[0][predicted_class_index]  # Extract confidence score
    # ------------------------------------------------------------
    # st.success(f'Predicted class index: {predicted_class_index} with confidence: {confidence_score * 100:.2f}%')
    # ------------------------------------------------------------
    predicted_class_name = class_indices.get(str(predicted_class_index), "Unknown")
    return predicted_class_name, confidence_score
```

Figure 6.7: User Image Preprocessing and Prediction

6.4.4 Algorithm 4: User Image Preprocessing

This algorithm is used post-training to classify new images. It loads the trained model and class index mappings from JSON. The uploaded image is preprocessed using the same pipeline as the training images, and then passed through the model to obtain prediction probabilities. The class with the highest probability is selected, and the corresponding class name is displayed along with Prescription generated by Gemini AI.



```python
# Function detect largest leaf then crop it and removes background
def detect_and_crop_leaf(image: Image.Image):...

# Function to Load and Preprocess the Image using Pillow
def load_and_preprocess_image(image_path, target_size=(256, 256)):  1 usage
    # ------------------------------------
    # Load the image
    img = Image.open(image_path)
    # Resize the image
    img = img.resize(target_size)
    # Convert the image to a numpy array
    img_array = np.array(img)
    # Add batch dimension
    img_array = np.expand_dims(img_array, axis=0)
    # Scale the image values to [0, 1]
    img_array = img_array.astype('float32') / 255.
    return img_array
    # ------------------------------------------------

# Function to Predict the Class of an Image
def predict_image_class(model, image_path, class_indices):...
```

Figure 6.8: User Image Preprocessing

CHAPTER 7
SOFTWARE TESTING

7.1 Type of Testing

- **Unit Testing:** Individual components such as image preprocessing functions and model loading functions were tested independently to ensure they function correctly.
- **Integration Testing:** This ensured that modules like the trained model, class mappings, and prediction pipeline work seamlessly when integrated.
- **Functional Testing:** Verified that the system accepts user-uploaded images, processes them correctly, and outputs the predicted plant disease.
- **Validation Testing:** Compared model predictions with ground truth data using validation accuracy, precision, and recall metrics.
- **User Acceptance Testing (UAT):** The complete system was tested in the Streamlit interface to ensure it met end-user expectations in terms of usability and accuracy.

7.2 Test Cases & Test Results

- **Test Case 1: Uploading and Preprocessing an Image**
  - **Input:** An image of a diseased plant leaf
  - **Expected Result:** Image is resized and normalized
  - **Actual Result:** Image processed successfully
  - **Status:** Pass

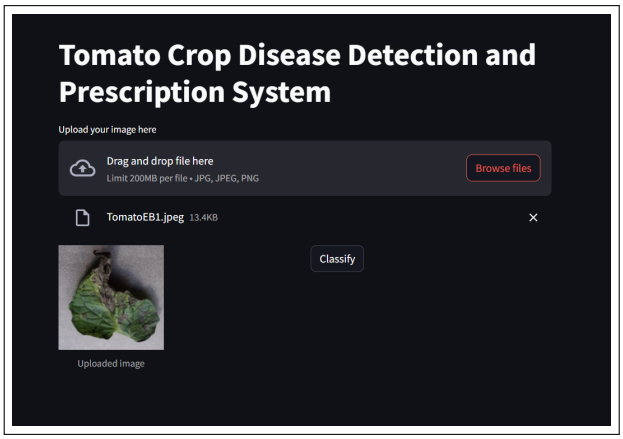


Figure 7.1: Upload Image Testing

- **Test Case 2: Model Prediction And Prescription**
  - **Input:** Tomato Early Blight Image form Imternet
  - **Expected Result:** Tomato Early Blight
  - **Actual Result:** Tomato Early Blight
  - **Status:** Pass

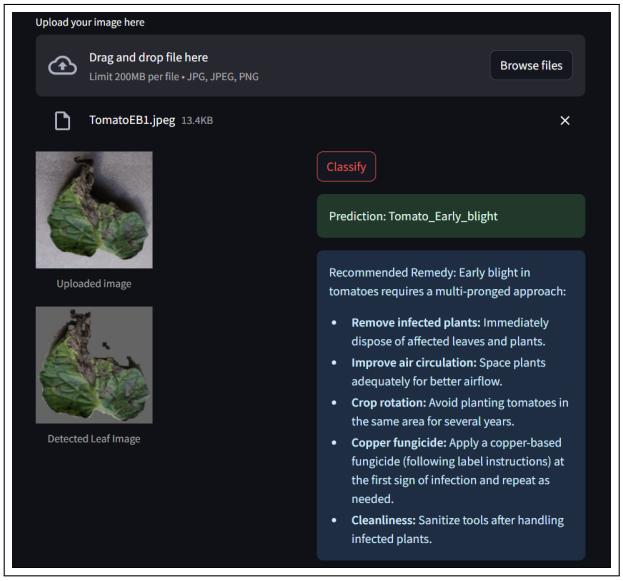


Figure 7.2: Prediction and Prescription

- **Test Case 3: Streamlit UI Functionality**
  - **Input:** User uploads an image
  - **Expected Result:** Model prediction and class displayed with confidence
  - **Actual Result:** Output displayed successfully
  - **Status:** Pass
- **Performace Metrics**

```
results = model.evaluate(
    validation_generator,
    steps=validation_generator.samples // batch_size,
    return_dict=True
)
print(f"Validation Accuracy: {results['accuracy'] * 100:.2f}%")
print(f"Validation Precision: {results['Precision'] * 100:.2f}%")
print(f"Validation Recall: {results['Recall'] * 100:.2f}%")
print(f"Validation Loss: {results['loss']:.4f}")
```

```
Evaluating model...
549/549 ━━━━━━━━━━━━━━━━━━━━ 32s 58ms/step - Precision: 0.9751 - Recall: 0.9696 - accuracy: 0.9720 - loss: 0.2810
Validation Accuracy: 97.23%
Validation Precision: 97.60%
Validation Recall: 97.04%
Validation Loss: 0.2814
```

Figure 7.3: Model Performance Score

CHAPTER 8
CONCLUSIONS

8.1 Conclusion

This project demonstrates an effective system for automated crop disease detection and prescription using image segmentation, deep learning, and generative AI. Implemented as a Streamlit web application, it allows users to upload leaf images, automatically detect the leaf region, classify the disease using a CNN, and receive treatment recommendations generated by a language model.

The system addresses the limitations of manual inspection by offering rapid, consistent, and accessible diagnosis—crucial for timely intervention in agriculture. Its high performance underscores its practical utility:

- **Accuracy:** 97.23%
- **Precision:** 97.60%
- **Recall:** 97.04%
- **Loss:** 0.2814

These results validate the model's robustness and suitability for real-world deployment. Future work will focus on supporting more crops, enhancing mobile usability, and enabling offline access, thereby making the tool even more impactful for farmers and agricultural professionals.

8.2 Future Scope

The system developed in this project can be expanded and enhanced in several impactful ways:

- **Multi-Crop Support:** Extend the system to detect diseases in other crops such as rice, wheat, and cotton by training on diverse datasets.
- **Mobile Application Integration:** Deploy the solution as a mobile app to reach farmers in remote and rural areas with limited access to computers.
- **Real-time Detection:** Integrate with drone or IoT-based cameras for real-time disease monitoring in large agricultural fields.
- **Multilingual Support:** Include recommendations in regional languages to ensure accessibility for non-English-speaking farmers.

8.3 Applications

This system can have wide-ranging practical uses in the agricultural domain:

- **Farm-Level Disease Diagnosis:** Enable farmers to identify crop diseases early and accurately using only a smartphone camera.
- **Agricultural Extension Services:** Assist extension workers in quickly diagnosing plant issues in the field and prescribing standardized remedies.
- **Precision Agriculture:** Support more efficient pesticide and nutrient use by identifying affected regions and applying localized treatment.
- **Smart Farming Systems:** Integrate into autonomous systems that monitor plant health and optimize field operations.

CHAPTER 9
APPENDIX

9.1 Appendix A

9.1.1 Survey Paper Details

Paper Status: Published

Name of Journal: Tomato crop disease detection and prescription using CNN

9.2 Appendix B

9.2.1 Research Paper Details

Paper Status: Yet to Submit

Name of Journal: CNN Model for Tomato Leaf Disease Detection and Prescription

9.3 Appendix C

9.3.1 Plagiarism Report

The plagiarism report for the project report has been attached herewith.
Similarity: %

CHAPTER 10
REFERENCES

[1] Agarwal, A., Sarkar, A., Dubey, A.K. (2019). Computer Vision-Based Fruit Disease Detection and Classification. In: Tiwari, S., Trivedi, M., Mishra, K., Misra, A., Kumar, K. (eds) Smart Innovations in Communication and Computational Sciences. Advances in Intelligent Systems and Computing, vol 851. Springer, Singapore. https://doi.org/10.1007/978-981-13-2414-7S-11

[2] Jafar A, Bibi N, Naqvi RA, Sadeghi-Niaraki A and Jeong D (2024) Revolutionizing agriculture with artificial intelligence: plant disease detection methods, applications, and their limitations. Front. Plant Sci. 15:1356260. doi: 10.3389/fpls.2024.1356260 https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2024.1356260/full

[3] Ding, J., Qiao, Y. Zhang, L. Plant disease prescription recommendation based on electronic medical records and sentence embedding retrieval. Plant Methods 19, 91 (2023). https://doi.org/10.1186/s13007-023-01070-6

[4] Li L, Zhang S, Wang B. "Plant disease detection and classification by deep learning—a review. IEEE Access. 2021;9:56683–98. https://doi.org/10.1109/ACCESS.2021.3069646.

[5] Ding, J., Li, B., Xu, C. et al. Diagnosing crop diseases based on domain-adaptive pre-training BERT of electronic medical records. Appl Intell 53, 15979–15992 (2023). https://doi.org/10.1007/s10489-022-04346-x

[6] Alzubaidi, L., Zhang, J., Humaidi, A.J. et al. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. J Big Data 8, 53 (2021). https://doi.org/10.1186/s40537-021-00444-8

[7] Kh. Nafizul Haque What is Convolutional Neural Network — CNN (Deep Learning) April 3, 2023. https://www.linkedin.com/pulse/what-convolutional-neural-network-cnn-deep-learning-nafiz-shahriar/

[8] Wang, X., Liu, J. An efficient deep learning model for tomato disease detection. Plant Methods 20, 61 (2024). https://doi.org/10.1186/s13007-024-01188-1

[9] Liu, J., Wang, X. Early recognition of tomato gray leaf spot disease based on MobileNetv2-YOLOv3 model. Plant Methods 16, 83 (2020). https://doi.org/10.1186/s13007-020-00624-2

[10] Zhang, J., Qi, C., Mecha, P. et al. Pseudo high-frequency boosts the generalization of a convolutional neural network for cassava disease detection. Plant Methods 18, 136 (2022). https://doi.org/10.1186/s13007-022-00969-w

[11] Naresh K.Trivedi , Vinay Gautam, Abhineet Ananad, Hani Moaiteq Aljahdali, Santos Gracia Villar, Divya Anand, Nitin Goyal Seifedine Kadry (2021). "Early Detection and Classification of Tomato Leaf Disease Using High-Performance Deep Neural Network." Sensors 2021, 21(23), 7987;

[12] Mohit Agarwal , Suneet Kr. Gupta K.K. Biswas (2020). "Development of Efficient CNN model for Tomato crop disease identification." Sustainable Computing: Informatics and Systems Volume 28, December 2020, 100407

[13] Alex Krizhevsky, Ilya Sutskever Geoffrey E. Hinton (2012)."ImageNet Classification with Deep Convolutional Neural Networks."

[14] Sharada P. Mohanty, David P. Hughes Marcel Salathe (2016)."Using Deep Learning for Image-Based Plant Disease Detection." Frontiers in Plant Science, 7, 1419.

[15] Kim, Sam-Keun Ahn, Jae-Geun(2021)."Tomato Crop Diseases Classification Models Using Deep CNN-based Architectures." Journal of the Korea Academia-Industrial cooperation Society Volume 22 Issue 5 Pages.7-14 / 2021 / 1975-4701(pISSN) / 2288-4688(eISSN)

[16] Chang Xu , Junqi Ding, Yan Qiao Lingxian Zhang(2022)."Tomato disease and pest diagnosis method based on the Stacking of prescription data." Computers and Electronics in Agriculture Volume 197, June 2022, 106997

[17] Siventhirarajah Sangeevan. "Deep learning-based pesticides prescription system for leaf diseases of home garden crops in Sri Lanka." 2021 International Research Conference on Smart Computing and Systems Engineering (SCSE), 2021,2612-8662.

[18] Marko Arsenovic,Mirjana Karanovic,Srdjan Sladojevic,Andras Anderla Darko Stefanovic ."Solving Current Limitations of Deep Learning Based Approaches for Plant Disease Detection." Symmetry 2019, 11(7), 939